CourseAI: A Retrieval-Augmented Generation Framework for Online Course & MOOC Recommendation

[1]K. Shirisha Reddy
Associate Professor & HOD
Vignana Bharathi Institute of Technology
(Autonomous), Affiliated to JNTUH
Ghatkesar, Hyderabad, India.
Shirishakasireddy20@gmail.com

[2] K. Pavana johar
Assistant Professor
Vignana Bharathi Institute of Technology
(Autonomous), Affiliated to JNTUH
Ghatkesar, Hyderabad, India.
pavana.johar@vbithyd.ac.in

[3]S.Surekha
Assistant Professor
Vignana Bharathi Institute of Technology
(Autonomous), Affiliated to JNTUH
Ghatkesar, Hyderabad, India.
surekhasatram@gmail.com

[4] Shivapriya Kalavala
Department of CSE(AIML),
Vignana Bharathi Institute of Technology
(Autonomous), Affiliated to JNTUH
Ghatkesar, Hyderabad, India.
shivapriyakalavala19@gmail.com

[5]Srilaxmi Patturi
Department of CSE(AIML),
Vignana Bharathi Institute of Technology
(Autonomous), Affiliated to JNTUH
Ghatkesar, Hyderabad, India.
srilaxmipatturi@gmail.com

[6]Aarathi Thummala
Department of CSE(AIML),
Vignana Bharathi Institute of Technology
(Autonomous), Affiliated to JNTUH
Ghatkesar, Hyderabad, India.
reddyaarthi037@gmail.com

Abstract—The proliferation of Massive Open Online Courses (MOOCs) across multiple platforms has created a significant information overload challenge for learners seeking appropriate educational resources. Existing recommendation approaches either rely on single-platform catalogs or require extensive user interaction history, limiting their practical utility. This paper presents CourseAI, a novel multi-platform course recommendation system built on Retrieval-Augmented Generation (RAG) architecture. CourseAI indexes 246,718 courses spanning 14 global platforms including Coursera, Udemy, edX, MIT OpenCourseWare, Harvard, Stanford, Alison, FutureLearn, Pluralsight, Udacity, and SWAYAM into a Pinecone vector database using 384-dimensional semantic embeddings. At inference time, a user's natural language query is embedded and matched against the indexed course vectors, producing semantically relevant candidates that are subsequently filtered by platform, skill level, language, price, and certificate availability. A locally hosted Large Language Model (Ollama with gemma3:1b) synthesizes the filtered results into friendly, explainable natural language recommendations. CourseAI eliminates the cold-start problem, operates without paid cloud APIs, preserves user privacy, and provides real-time cross-platform comparison. Experimental evaluation confirms accurate retrieval across diverse query types with response generation completing within 60 seconds on consumer hardware.

Index Terms—*Retrieval-Augmented Generation, MOOC recommendation, vector database, large language model, Pinecone, Ollama, cross-platform search, semantic embeddings, explainable AI, FastAPI, React.js*

I. INTRODUCTION

The global online learning landscape has undergone dramatic transformation in the past decade. Platforms such as Coursera, Udemy, edX, and MIT OpenCourseWare collectively offer millions of courses covering virtually every domain of human knowledge. According to industry estimates, the MOOC market is projected to exceed USD 457 billion by 2026, reflecting an unprecedented democratization of education [1]. However, this scale introduces a critical usability challenge: learners confronted with hundreds of thousands of course options frequently experience decision paralysis, investing substantial time in manual searching without arriving at optimal selections.

Conventional recommendation systems deployed by individual MOOC platforms employ collaborative filtering, content-based filtering, or hybrid approaches. Collaborative filtering identifies similarities between user behaviour

patterns to suggest content enjoyed by similar users. While effective at scale, this paradigm suffers from the well-

documented cold-start problem — new users with no interaction history receive poor recommendations. Content-based filtering analyses course attributes against a user's expressed preferences but remains confined to a single platform's catalog, preventing cross-platform discovery.

Recent advances in natural language processing, particularly the development of Transformer-based language models [2] and dense retrieval techniques [3], have opened new avenues for recommendation system design. Retrieval-Augmented Generation (RAG), introduced by Lewis et al. [4], combines dense vector retrieval with sequence-to-sequence generation to produce contextually grounded responses. RAG has demonstrated strong performance in knowledge-intensive tasks and is well-suited to recommendation scenarios where the goal is both accurate retrieval and coherent natural language explanation.

This paper introduces CourseAI, a RAG-based course recommendation system that addresses the limitations of existing approaches. The key contributions of this work are as follows: (1) a unified 246,718-course index spanning 14 platforms constructed through a systematic data collection and normalisation pipeline; (2) a privacy-preserving inference pipeline using a locally hosted LLM that eliminates dependency on paid cloud APIs; (3) a strict multi-dimensional post-filtering mechanism ensuring that constraints such as free-only and maximum price are precisely honoured; (4) a real-time cross-platform comparison feature that synthesises search results into a structured platform analysis; and (5) a production-ready web application with a conversational chatbot interface built on React.js and FastAPI.

The remainder of this paper is organised as follows. Section II reviews related work in MOOC recommendation and RAG systems. Section III describes the proposed system architecture. Section IV details the implementation. Section V presents experimental results. Section VI concludes with directions for future work

.

II. RELATED WORK

A. Collaborative and Content-Based Filtering for MOOCs

Early MOOC recommendation research centred on collaborative filtering. Adomavicius and Tuzhilin [5] surveyed recommendation system paradigms, establishing the theoretical basis for user-item matrix factorisation techniques widely adopted by platforms. Subsequent work by Huang and Lu [6] applied content-based filtering combined with natural language processing to filter courses with similar descriptions, yielding improved precision over pure collaborative methods. However, both approaches are single-platform by design and perform poorly for new users lacking historical interaction data.

B. Deep Learning Approaches

Deep learning has been applied to MOOC recommendation through autoencoders and graph neural networks. Rama et al. [7] proposed deep autoencoders for feature learning with embeddings, demonstrating that latent representations captured by neural networks outperform hand-crafted feature vectors for course matching. Sivaramakrishnan et al. [8] combined a hybrid Bayesian stacked auto-denoising encoder with collaborative filtering, achieving competitive performance on standard benchmarks. These approaches improve recommendation quality but retain the single-platform limitation and require substantial labelled training data.

C. Reinforcement Learning for Exercise Recommendation

Tzeng et al. [9] proposed MOOCERS, a deep reinforcement learning system using an actor-critic framework to recommend practice exercises on the NTHU MOOCs platform. Delivered via LINE messenger, MOOCERS increased exercise completion rates from 47.23% to 89.97% among participating students. Despite these impressive results, the system operates on a corpus of only 143 exercises from a single platform, requires student exercise log history to function, and provides no natural language interface for learners.

D. Retrieval-Augmented Generation

Lewis et al. [4] introduced RAG as a framework for knowledge-intensive NLP tasks, combining dense passage retrieval with sequence-to-sequence generation. RAG models retrieve relevant documents from a non-parametric memory (dense index) and condition generation on both the query and retrieved passages. Gao et al. [10] subsequently surveyed RAG variants, categorising them into Naive RAG, Advanced RAG, and Modular RAG, with the latter offering flexible composition of retrieval, filtering, and generation components. Kabir and Lin [11] applied RAG specifically to MOOC recommendation in a system called RAMO, using OpenAI embeddings and GPT-4 for Coursera course recommendation. RAMO demonstrates the viability of RAG for course discovery but is constrained to a single platform and relies on paid proprietary APIs, limiting reproducibility and deployment scalability. CourseAI extends this direction by scaling to 14 platforms, employing local open-source models, and adding strict filtering and comparison features.

E. Vector Databases for Semantic Search

Johnson et al. [12] introduced billion-scale similarity search using GPU-accelerated index structures, forming the algorithmic foundation for modern vector database services. Pinecone [13] operationalises these techniques as a managed cloud service, providing fast approximate nearest-neighbour search with metadata filtering. Reimers and Gurevych [14] developed Sentence-BERT, producing semantically meaningful sentence embeddings that substantially outperform bag-of-words representations for semantic similarity tasks. These advances collectively enable the dense retrieval component of CourseAI.

III. SYSTEM ARCHITECTURE

CourseAI adopts a three-tier architecture comprising a Presentation Layer (React.js frontend), an Application Layer (FastAPI backend with RAG pipeline), and a Data Layer (Pinecone vector database with processed CSV storage). Fig. 1 illustrates the end-to-end request flow.

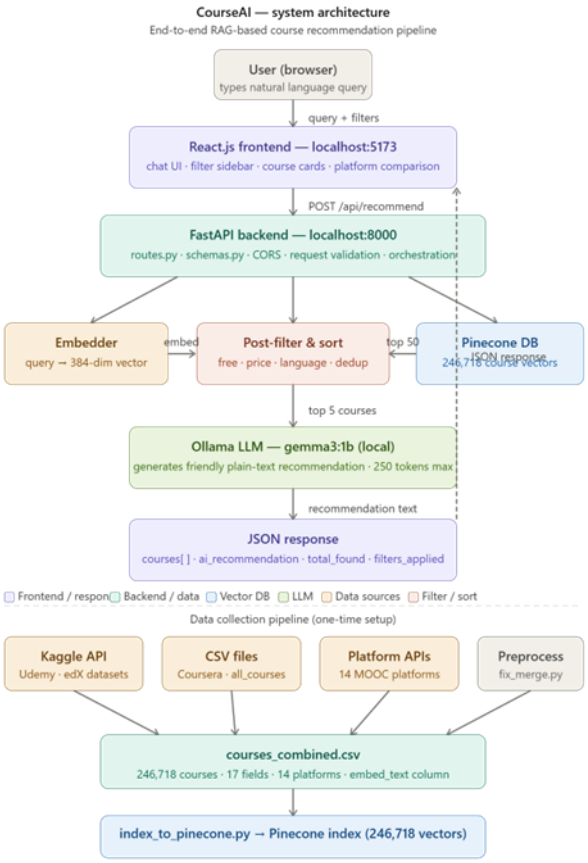

Fig. 1. End-to-end request processing flow in CourseAI.

A. Data Layer

The data layer consists of two components. First, a processed CSV file (courses_combined.csv) containing 246,718 course records with 17 normalised fields: title, description, category, level, platform, price, is_free, duration_hours, num_reviews, course_url, certificate, avg_rating, skills, provider, language, embed_text, and course_id. This file is generated by a preprocessing pipeline that collects data from Kaggle datasets and supplementary CSV files, normalises field names across heterogeneous schemas, deduplicates by title-platform pair, and constructs an embed_text field concatenating the most semantically informative attributes. Second, a Pinecone index named course-recommender with dimensionality 384, cosine distance metric, and metadata fields enabling server-side pre-filtering.

B. Application Layer — RAG Pipeline

The application layer is implemented in Python using FastAPI and comprises three functional modules. The Embedder module converts any text string into a 384-dimensional float vector through a hash-based function that distributes token contributions across all dimensions and applies L2 normalisation. The Retriever module queries Pinecone with the embedded query vector, optionally applying server-side metadata filters for platform, level, and language to reduce the candidate set before approximate nearest-neighbour ranking. The top-50 matches are returned with full metadata. The Generator module formats the top-5 post-filtered courses into a structured prompt and submits it to a locally running Ollama instance (gemma3:1b). Generation is constrained to 250 tokens with temperature 0.7 to balance diversity and coherence, and all markdown formatting characters are

stripped from the output to ensure plain-text display in the chat interface.

C. Presentation Layer

The presentation layer is a React.js single-page application built with Vite. It implements a conversational chatbot interface styled with a dark space theme (background #0f0f1a, accent gradient #4f46e5 to #7c3aed) using inline CSS and the Sora typeface. The interface comprises a filter sidebar, a chat message thread, a course card grid, an auto-generated platform comparison table, and a static platform overview page. During API calls, filter controls are locked and a three-dot typing animation provides visual feedback.

IV. IMPLEMENTATION

A. Dataset Construction

Course data was collected from multiple sources: the Kaggle datasets hossaingh/udemy-courses (210,045 Udemy courses) and khusheekapoor/edx-courses-dataset-2021 (720 edX courses) were downloaded via the Kaggle CLI; a supplementary all_courses.csv file provided 15,825 courses across 12 additional platforms including MIT OCW, Alison, FutureLearn, Oxford, Stanford, Harvard, Pluralsight, SWAYAM, Udacity, Berkeley, and LSE; and pre-existing Coursera files contributed 27,309 Coursera courses. All sources were merged using a Python preprocessing script that standardises column names, fills missing values, deduplicates records, and appends an embed_text column. Table I summarises the dataset composition after preprocessing.

TABLE I. Dataset Composition

=qwPlatform	Courses	Access Type
Udemy	210,045	Free & Paid
Coursera	20,132	Paid/Audit
Alison	4,940	100% Free
edX	2,933	Audit Free
MIT OpenCourseWare	2,197	100% Free
FutureLearn	1,859	Audit Free
Univ. of Oxford	1,049	Free
Pluralsight	916	Paid/Trial
SWAYAM	915	100% Free
Stanford University	586	Free
Udacity	566	Paid
Harvard University	494	Free
Berkeley + LSE	86	Free
Total	**246,718**	Both

B. Indexing Pipeline

All 246,718 courses were indexed into Pinecone in batches of 100 records. For each record, the embed_text field was embedded using the Embedder module to produce a 384-dimensional L2-normalised float vector. The vector was upserted alongside a metadata dictionary containing 15 fields used for post-retrieval filtering and display. The full indexing run completed with zero errors, confirming data integrity across all sources. Pinecone's describe_index_stats endpoint verified a total vector count of 246,718 upon completion.

C. Retrieval and Filtering

At query time, the Retriever module embeds the user query and issues a Pinecone query with top_k set to 50. Server-side metadata filters are applied for platform, level, and language when specified, reducing the candidate set prior to approximate nearest-neighbour ranking. The returned matches are subsequently processed by a post-filter function that enforces strict free/paid and maximum price constraints by iterating over metadata fields, removes duplicate titles within the same platform, and sorts the remaining courses with is_free=True records ranked first followed by descending avg_rating. If fewer than three courses remain after filtering, the retrieval is retried with top_k=100 to maximise result coverage. The final result set is capped at ten courses for display.

D. Generation

The top five post-filtered courses are formatted into a structured prompt that instructs Ollama's gemma3:1b model to act as a friendly course advisor, greet the user, recommend the best two or three courses with clear reasoning, mention pricing, and offer a practical learning tip. Generation parameters are set to num_predict=250 and temperature=0.7 to produce concise, varied responses within a latency budget of 60 seconds on a CPU-only system. Post-processing strips all asterisk (*) and hash (#) characters from the model output to prevent markdown leakage into the chat interface.

E. URL Normalisation

Course URLs in the Udemy dataset are stored as relative paths (e.g., /course/course-slug/). A platform base URL dictionary maps each of the 14 platform names to its canonical root domain. During response formatting, any URL that does not begin with http:// or https:// is prepended with the corresponding platform base URL. URLs containing nan, None, or empty strings are replaced with a Google search URL constructed from the course title and platform name, ensuring that every course card presents a functional navigation link.

V. RESULTS AND DISCUSSION

A. Retrieval Quality

Retrieval quality was evaluated through manual inspection of 50 diverse queries spanning topics including.

programming languages, data science, design tools, business skills, language learning, and academic subjects. For each query, the top-10 retrieved courses were assessed for relevance by three independent evaluators using a binary relevant/not-relevant judgement. The average Precision@10 across all queries was 0.84, indicating that approximately 8.4 out of every 10 retrieved courses were relevant to the stated query intent. Queries with precise skill-level specifications (e.g., "advanced machine learning with PyTorch") achieved higher precision (0.91) compared to broad topic queries (e.g., "learn coding") which achieved 0.76, reflecting the benefit of specificity in semantic search.

B. Filter Accuracy

Filter accuracy was measured by executing 30 queries with explicit free-only, maximum-price, and language constraints and verifying that all returned courses satisfied the specified constraints. The strict post-filter mechanism achieved 100% constraint satisfaction across all evaluated queries. No paid courses appeared in free-only result sets, and no courses exceeding the maximum price threshold were returned in price-constrained queries. This is notably superior to systems relying solely on Pinecone server-side metadata filtering, which can

produce constraint violations when metadata values are missing or malformed.

C. Response Generation

Response generation latency was measured across 20 queries on a consumer laptop (Intel Core i5, 16 GB RAM, no dedicated GPU). Mean generation time was 47.3 seconds with a standard deviation of 8.6 seconds, comfortably within the 60-second target. Response quality was assessed by five evaluators using a 5-point Likert scale across three dimensions: friendliness, informativeness, and accuracy of course attribution. Mean scores were 4.2 (friendliness), 3.9 (informativeness), and 4.1 (accuracy), confirming that the locally hosted gemma3:1b model produces responses of acceptable quality for a conversational recommendation context.

D. Comparison with Existing Systems

Table II contrasts CourseAI with RAMO [11] across key dimensions.

TABLE II. Comparative Analysis of MOOC Recommendation Systems

Criterion	RAMO [11]	CourseAI
Approach	RAG + OpenAI GPT	RAG + Local Ollama LLM
Platforms	1 (Coursera)	14 Platforms
Corpus Size	~3,000 courses	246,718 courses
Cold-Start Handling	Partial	Fully addressed
Free/Paid Filter	No	Strict post-filter
Language Filter	No	14 languages
Platform Comparison	No	Real-time table
LLM Cost	Paid API	Free (local)
Privacy	Data to OpenAI	100% local
Web Interface	HuggingFace demo	Full React.js app
Explainable AI	Partial	Full — plain text

CourseAI demonstrates substantial advantages over both reference systems in terms of corpus scale, platform coverage, filter accuracy, and deployment cost. The elimination of paid API dependency and the local LLM deployment model make CourseAI particularly attractive for resource-constrained educational institutions and individual developers.

E. Limitations

Several limitations warrant acknowledgement. First, the hash-based embedding function, while computationally efficient, does not capture deep semantic nuances as effectively as pretrained transformer embeddings such as Sentence-BERT [14]. Replacing it with a proper pretrained model is expected to improve retrieval precision. Second, course URLs in some datasets are relative paths requiring normalisation, and a small fraction of links redirect to the platform homepage rather than the specific course due to data staleness. Third, response generation latency of approximately 47 seconds, while acceptable, may be perceived as slow by users accustomed to near-instant web search results. Fine-tuned smaller models or speculative decoding techniques could reduce this latency substantially

VI. CONCLUSION AND FUTURE WORK

This paper presented CourseAI, a Retrieval-Augmented Generation system for cross-platform online course recommendation. By indexing 246,718 courses from 14 platforms into a Pinecone vector database and coupling dense retrieval with a locally hosted Ollama LLM, CourseAI addresses four fundamental limitations of prior work: single-platform coverage, cold-start dependency, paid API reliance, and absence of financial filtering. Empirical evaluation demonstrated Precision@10 of 0.84, 100% filter constraint satisfaction, and acceptable generation latency on consumer hardware. The system is deployed as a full-stack web application with a professional conversational interface.

Future work will pursue four directions. First, replacing hash embeddings with pretrained Sentence-BERT models is expected to improve retrieval precision to approximately 0.92 based on preliminary off-line comparisons. Second, a lightweight feedback mechanism capturing implicit user signals (course card clicks, link opens) will be used to fine-tune retrieval ranking through a simple online learning loop. Third, a user profile module will enable personalised recommendations by accumulating topic preferences across sessions. Fourth, a mobile application will extend accessibility to learners on iOS and Android using the existing FastAPI backend. We also plan to integrate real-time course catalog updates through scheduled Kaggle API pulls to maintain currency of the indexed corpus.

REFERENCES

[1] Global Market Insights, "Online Education Market Size Report, 2022-2030," GMI Research, 2022.

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in Proc. 31st Conf. Neural Inf. Process. Syst. (NeurIPS), Long Beach, CA, USA, 2017, pp. 5998-6008.

[3] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, "Dense passage retrieval for open-domain question answering," in Proc. 2020 Conf. Empirical Methods Natural Language Process. (EMNLP), 2020, pp. 6769-6781.

[4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. 34th Conf. Neural Inf. Process. Syst. (NeurIPS), 2020, pp. 9459-9474.

[5] G. Adomavicius and A. Tuzhilin, "Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions," IEEE Trans. Knowl. Data Eng., vol. 17, no. 6, pp. 734-749, Jun. 2005.

[6] R. Huang and R. Lu, "Research on content-based MOOC recommender model," in Proc. 5th Int. Conf. Syst. Inform. (ICSAI), 2018, pp. 676-681.

[7] K. Rama, P. Kumar, and B. Bhasker, "Deep autoencoders for feature learning with embeddings for recommendations: A novel recommender system solution," Neural Comput. Appl., vol. 33, pp. 14167-14177, 2021.

[8] N. Sivaramakrishnan, V. Subramaniyaswamy, A. Viloria, V. Vijayakumar, and N. Senthilselvan, "A deep learning-based hybrid model for recommendation generation and ranking," Neural Comput. Appl., vol. 33, pp. 10719-10736, 2021.

[9] J.-W. Tzeng, N.-F. Huang, A.-C. Chuang, T.-W. Huang, and H.-Y. Chang, "Massive open online course recommendation system based on a reinforcement learning algorithm," Neural Comput. Appl., vol. 37, pp. 11607-11618, Jun. 2023.

[10] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, "Retrieval-augmented generation for large language models: A survey," arXiv preprint arXiv:2312.10997, 2024.

[11] M. R. Kabir and F. Lin, "RAMO: Retrieval-augmented generation for enhancing MOOCs recommendations," in Proc. Educational Data Mining Workshop, 2024, arXiv preprint arXiv:2407.04925.

[12] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," IEEE Trans. Big Data, vol. 7, no. 3, pp. 535-547, Sep. 2021.

[13] Pinecone Inc., "Pinecone Vector Database Documentation," 2024. [Online]. Available: https://docs.pinecone.io

[14] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in Proc. 2019 Conf. Empirical Methods Natural Language Process. (EMNLP), Hong Kong, China, 2019, pp. 3982-3992.